Machine Learning (2026) 115:254
https://doi.org/10.1007/s10994-026-07188-7

PoAC: Problem-Oriented AutoML in Clustering

Matheus Camilo da Silva[1,2] · **Gabriel Marques Tavares**[3,4] · **Eric Medvet**[2] · **Sylvio Barbon Junior**[2]

Received: 27 September 2024 / Revised: 7 October 2026 / Accepted: 9 October 2026
© The Author(s), under exclusive licence to Springer Science+Business Media LLC, part of Springer Nature 2026

Abstract
This work presents the *Problem-oriented AutoML in Clustering* (PoAC) framework, a flexible approach to automating clustering tasks that addresses key limitations of existing AutoML solutions. Traditional methods typically rely on fixed sets of meta-features and predefined internal Cluster Validity Indices (CVIs), limiting their adaptability across different problem contexts. PoAC introduces a modular design in which the core methodology, comprising meta-feature extraction, surrogate model training, and optimization, is fixed, while the specific configuration of components is user-defined. This design allows users to dynamically instantiate PoAC for different clustering goals by selecting which meta-features to extract, which CVIs to optimize, and how to structure the learning task. The framework is algorithm-agnostic and can adapt to diverse clustering problems without retraining from scratch. In the visualization-oriented study, PoAC obtained the highest mean ARI and SIL and the lowest mean DBS among the evaluated AutoML frameworks. In the anomaly-detection study, it obtained higher F1 scores than IForest on most of the eight datasets and broadly comparable ROC AUC values. These findings are limited to the reported datasets, baselines, search spaces, and computational budgets.

Keywords Clustering · Anomaly detection · Unsupervised learning · AutoML · Pipeline synthesis · Surrogate model · Meta-features · Meta-learning

1 Introduction

Most Machine Learning (ML) applications are composed by a sequence of steps (i.e., a pipeline) involving the selection of algorithms and the setting of their hyperparameters. However, carefully designing good performing pipelines is a tedious, error-prone task that often relies on expert knowledge, which is frequently unavailable or expensive (Olson et al., 2016). Automated Machine Learning (AutoML) provides the tools to make the creation and configuration of optimal ML solutions more accessible to practitioners with varying levels

Editor: Michelangelo Ceci and Xiaoli Fern.

Extended author information available on the last page of the article

of expertise by automating the complex and time-consuming aspects of ML development (Hutter et al., 2019).

This automation is particularly straightforward for supervised tasks where labeled data is available, allowing for objective evaluation metrics like accuracy, precision, or F1 score to guide optimization. By systematically testing different pipeline configurations, AutoML can identify the most effective solutions for a given dataset (He et al., 2021; Truong et al., 2019).

In the case of unsupervised tasks, the lack of labels, pose a challenge for the evaluation of generated pipelines. A conventional approach is the reliance on internal CVIs to synthesize and optimize clustering solutions (Bahri et al., 2022), while ignoring any external information of the clustering problem. This approach's only concern is to optimize CVIs (Kryszczuk & Hurley, 2010; Marutho et al., 2018; Tschechlov et al., 2021). This practice, however, can be limiting as each internal score evaluates only a facet of the relationships between data points in a clustering setting. Moreover, different evaluation metrics may lead to different conclusions about the quality of a clustering solution (Rokach & Maimon, 2005). For this reason, properly defining the clustering problem (i.e., the goal of the clustering analysis) and a corresponding optimization function is crucial for generating high quality automated clustering solutions (Luxburg et al., 2012).

Another approach in AutoML for clustering is to create solutions based on meta-learning, focusing on leveraging prior experience to enable models to quickly adapt to new tasks by learning from past clustering problems and using this knowledge to inform future decisions (Brazdil et al., 2022; Hutter et al., 2019). By accumulating insights from diverse datasets, meta-learning provides a way to transfer knowledge, thus reducing the time and computational resources needed to optimize new tasks. However existing works, as those by Poulakis et al. (2020); Liu et al. (2021); ElShawi et al. (2021); Treder-Tschechlov et al. (2023), tend to restrict their application of meta-learning by attaching a fixed set of algorithms, hyperparameter configurations, and CVIs to their meta-database, which can hinder generalization. This presents an opportunity for more flexible solutions that can adapt across a wider variety of clustering tasks and optimization objectives.

Since there is not a universal metric capable of describing every clustering goal equally well (Bonner, 1964), the definition of clustering quality can be subjective and dependent on the context of the problem that the practitioners conducting the clustering aim to solve (Bonner, 1964). That is, the clear association between the clustering task at hand and suitable metrics to measure performance should be considered when developing clustering solutions (Hennig, 2015; Luxburg et al., 2012; Van Mechelen et al., 2023). For instance, a partitioning that is optimal for data visualization could be inadequate for noise reduction.

Our proposed PoAC framework introduces a methodology that directly addresses the limitations of traditional AutoML solutions by establishing a dynamic connection between the clustering problem, CVIs, and meta-features. PoAC allows flexibility in adapting these components to the specific needs of the clustering task at hand. This flexibility enables users to customize both the meta-features and CVIs based on their problem's context, ensuring that the solution aligns with their particular objectives. At the core of PoAC there is a surrogate model, trained on a large meta-knowledge base of previous clustering datasets and their associated solutions. This surrogate model is then leveraged to infer the quality of new clustering pipelines, making it possible to synthesize optimal solutions for unseen datasets while accounting for the user's clustering goals.

This flexibility is a key differentiator of PoAC compared to traditional AutoML frameworks, which often offer generalized solutions with fixed CVIs and meta-features. While such general-purpose approaches aim to be widely applicable, they can become overly restrictive, providing solutions that only make sense in the context of the predefined CVIs they include. In contrast, PoAC's ability to adjust CVIs and meta-features ensures that the clustering pipeline is specifically tailored to the user's intent, thus delivering a more problem-oriented solution. By bridging the gap between the characteristics of the dataset and the quality metrics used to evaluate clustering, PoAC offers a more nuanced, data-driven approach to AutoML that reflects the practical goals of clustering practitioners. PoAC is algorithm-agnostic, generating solutions based on the available set of algorithms without demanding additional training or data ingestion. It is entirely decoupled from the optimization approach.

On the evaluated datasets, PoAC obtained the highest aggregate ARI and the most favorable SIL and DBS means among the compared clustering AutoML frameworks. The experiments also show that its optimizer can synthesize pipelines with or without preprocessing steps according to the dataset and configured objective.

To examine PoAC beyond visualization-oriented clustering, we instantiated it for global anomaly detection and compared it with Isolation Forest (IForest) on eight real-world UCI datasets. In this experiment, PoAC obtained higher F1 scores on most datasets and broadly comparable ROC AUC values. This additional use case demonstrates the framework's configurability, but does not by itself establish generalization to other anomaly types or unsupervised tasks.

We hypothesize that a surrogate-based, problem-oriented AutoML approach can outperform state-of-the-art unsupervised AutoML frameworks in both generalization and adaptability.

The remainder of this paper is structured as follows. Section 2 reviews the theoretical background and section 3, the related literature. Section 4 introduces the PoAC framework and details its methodological components. Section 5 and Section 6 present its application to two problem domains: clustering for data visualization and clustering for anomaly detection, respectively. Section 7 discusses the limitations of the proposed approach, and Section 8 concludes the paper and outlines future research directions.

2 Theoretical Background

AutoML refers to the application of techniques to automate the process of designing and optimizing ML pipelines (Hutter et al., 2019), i.e., acting in the intersection of automation and ML (Yao et al., 2019). Considering the plethora of algorithms and techniques for each step of a ML pipeline, the configuration space grows exponentially (Hutter et al., 2019). In practical scenarios, the abundance of options available for pipeline development introduces significant complexity, posing challenges even for seasoned experts. The conventional approach to this complexity involves iterative processes, frequently relying on trial-and-error methodologies (Zöller & Huber, 2021). Unfortunately, this process is inherently resource-intensive and time-consuming, often resulting in sub optimal pipelines due to the limited exploration of potential configurations.

AutoML literature has developed specific methods to solve different optimization problems, that is, breaking down the pipeline search to smaller, simpler tasks. The most elementary AutoML task is Algorithm Selection (AS). Given a portfolio of algorithms and a dataset, AS is the task of selecting the algorithm that maximizes performance for the input data. AS was first proposed and discussed by Rice (1976), where he formalizes the AS problem as the mapping of a *problem space* (represented by datasets for a specific problem) to a performance measure space that evaluates pipeline efficiency. This association might come in the form of meta-features describing the *problem space* behavior, therefore, mapping the *problem space* to a feature space, which, in turn, is correlated with an algorithm space containing potential solutions. The quality of algorithms is measured by the performance space. The ultimate goal is to find a mapping function that optimally associates problems with performances (going through algorithms), thereby maximizing performance for a given task.

Furthermore, algorithms are configured by their hyperparameters, which significantly affects performance in clustering settings (Mishra et al., 2022). In other words, optimizing the hyperparameters leads to improved generalizations and, consequently, better performances. This task is coined as Hyperparameter Optimization (HPO) (Hutter et al., 2019). HPO is conceptually similar to AS, however, hyperparameters often lie in continuous and high dimensional spaces, making the *search space* considerably larger. Hence, HPO is often more challenging than AS. While AS focuses on finding an optimal algorithm and HPO on selecting optimal hyperparameter configurations, both tasks can be joined. The Combined Algorithm Selection and Hyperparameter Optimization (CASH) task comprises both previously presented problems (Hutter et al., 2019). Although the *search space* is expanded even more, CASH solutions tend to outperform others as they combine important aspects of pipeline design that directly affect performance. As a natural extension, we also introduce here the Pipeline Synthesis (PS) task. Since CASH is only focused on algorithms and hyperparameters, PS extends the concept to additional pipeline steps, such as preprocessing, data transformation, feature selection, dimensionality reduction, among others. Therefore, in the scope of this research, PS refers to the search of optimal pipelines for a given clustering problem.

3 Related Works

The exploration of AutoML applications specifically tailored for clustering problems has gained some prominence (Souto et al., 2008; Ferrari & de Castro, 2015; Ferrari & Castro, 2012; Fernandes et al., 2021; Gabbay et al., 2021; Nascimento et al., 2009; Pimentel & de Carvalho, 2019; Pimentel & Carvalho, 2018, 2019; Soares et al., 2009). For that, many apply meta-learning techniques, which are known to perform quite well for AS. However, they are only able to select an algorithm out of a portfolio. Considering (i) the complexity of real environments (i.e., necessity of configuring several pipeline steps), (ii) the importance of tuning hyperparameters, specially for clustering, and (iii) that our solution searches for the complete pipeline, we limit the scope of our related work to research that tackles at least the CASH problem.

Poulakis et al. (2020) presented a framework for clustering problems based on a meta-learning approach. For this, the authors first build a meta-database containing information about datasets behavior (captured by meta-features) combined with CVIs measured after

the application of several clustering algorithms, which are known as landmarking meta-features. The meta-learning step recommends the algorithm to be used for a new dataset and is coupled with Bayesian optimization to tune the hyperparameters. Similarly, Liu et al. (2021) also builds a meta-database comprising clustering solutions and performance measurements. Moreover, the authors consider a multi-objective function as the optimization goal, which makes the problem more complex. Once the algorithm is selected, the framework applies grid search as a strategy for HPO. It has been pointed out, however, that grid search is the most naive optimization approach, often leading to non-optimal solutions (Hutter et al., 2019). ElShawi et al. (2021) proposed a solution that also uses meta-learning, but instead of only recommending the algorithm, it selects the appropriate CVI to be optimized considering dataset characteristics. Once algorithm and optimization goals are set, the method uses evolutionary algorithms for HPO. However, selecting different CVIs for each dataset leads to solutions that are not comparable as the goal is not unique. Finally, Cohen-Shapira and Rokach (2021) used meta-learning based on supervised graph embedding optimized towards clustering AS. The selected algorithm has its hyperparameters optimized using Bayesian optimization.

A common characteristic of previous solutions is the use of meta-learning for AS, which is in accordance with state-of-the-art in supervised AutoML. However, a clear limitation of such approaches is breaking down the CASH problem into first solving AS and, finally, HPO. Dividing CASH into two steps often leads to local optima, therefore, solutions might not achieve a suitable performance. This phenomenon is observed because once the search is narrowed down into only one algorithm, all the other candidate solutions in the portfolio are dropped down prematurely in the search. As clustering algorithms are heavily affected by hyperparameter configuration, algorithms that were ignored too early in the search might have achieved optimal performance after HPO.

A few solutions tackled the CASH problem directly (i.e., optimizing both algorithms and respective hyperparameters together) (Tschechlov et al., 2021; Treder-Tschechlov et al., 2023; Van Craenendonck & Blockeel, 2017). Van Craenendonck and Blockeel (2017) proposed a constraint-based approach where users input their restrictions and used grid search as the optimization mechanism. Tschechlov et al. (2021) introduced a framework with several optimization techniques such as random search, Bayesian optimization, and Hyperband. However, it is left to the user to choose the preferred technique along with the CVI to be optimized, going in the opposite direction of AutoML, where the aim is to release the burden of design choices from users. Finally, Treder-Tschechlov et al. (2023) also presented a framework based on meta-learning for warm-starting and CVI recommendation, which is then coupled with Bayesian optimization methods for HPO. However, as the authors showed, the clustering performance when evaluated with external CVIs (e.g., ARI, NMI) is poor. To the best of our knowledge, the only approach tackling the PS task was proposed by ElShawi and Sakr (2022). For that, the authors create a three-stepped framework: meta-learning for warm-starting, evolutionary algorithms for HPO, and clustering ensemble to provide a solution. However, the *search space* is quite limited, including only a few preprocessing techniques.

Although there are interesting approaches in the literature, specially from the engineering point of view, all of them ignore the theoretical limitations of proposing generic clustering solutions without considering the clustering goal. As pointed by Luxburg et al. (2012), one must consider the objective (i.e., for which reason the dataset is being clustered) in order

to build a clustering solution. The same applies for AutoML-based clustering frameworks, providing a fixed set of CVIs to be optimized for any given clustering task completely ignores the subjectivity inherently inserted in clustering analysis. Considering previous limitations, our approach is the first one that proposes a problem-oriented AutoML framework to synthesize clustering pipelines.

4 Problem-Oriented AutoML in Clustering (PoAC)

An overview of the proposed approach is displayed in Figure 1. It consists of four main stages: Problem Space Design, Feature Space Mapping, Surrogate Modeling, and Function Optimization. Each of these stages is discussed in detail below.

The **first stage** involves composing a problem space with labeled clustering datasets related to specific goals such as visualization, denoising, or dimensionality reduction. This foundational process aims to assemble a diverse collection of clustering problems that capture a wide range of structures, complexities, and data distributions. Together, these datasets form the basis for learning how different clustering characteristics relate to performance across various objectives.

In the **second stage**, the datasets are mapped to a meta-feature space, enabling PoAC to construct an enriched representation for surrogate model induction. Each dataset is described by unsupervised meta-features that capture its statistical, structural, and complexity-related properties, together with CVIs. To generate diverse CVI values, incremental

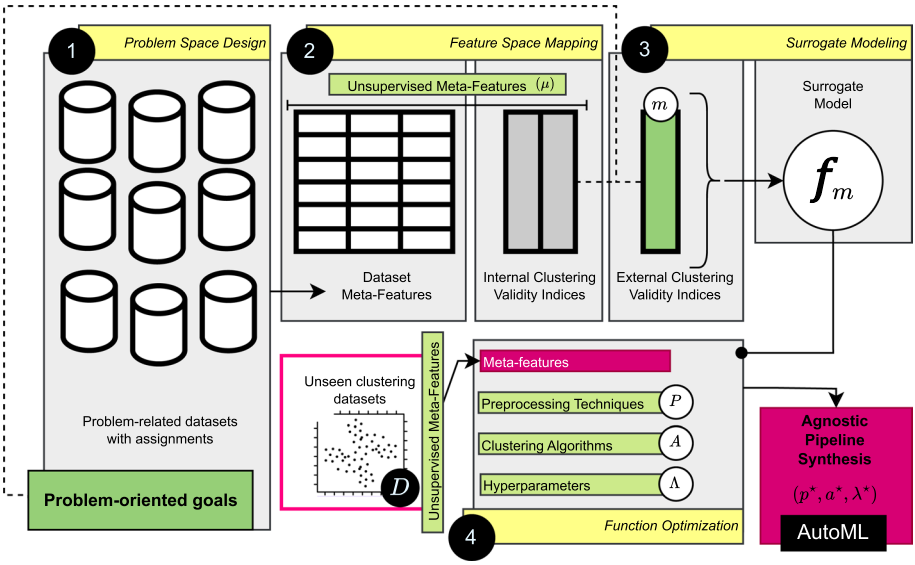


Fig. 1 Overview of the proposed Problem-oriented AutoML in Clustering (PoAC) framework. The system consists of four stages: (1) Problem Space Design — assembling labeled clustering datasets for specific goals (e.g., visualization, anomaly detection); (2) Feature Space Mapping — extracting unsupervised meta-features and internal CVIs; (3) Surrogate Modeling — training a predictive model to estimate external CVIs; and (4) Function Optimization — using the surrogate model as an objective function to synthesize optimal clustering pipelines. The process enables user-defined customization of CVIs and meta-features, ensuring problem-oriented adaptability

noise is added to the true cluster labels, simulating alternative partitions without rerunning clustering algorithms.

The **third stage** consists of training a surrogate model to predict an external CVI based on the extracted meta-features and internal CVIs. Various regression algorithms (or ensembles) can be employed for this purpose. The surrogate learns the relationships between dataset characteristics, clustering validation measures, and expected performance, forming a task-specific predictive model that will later guide the optimization process.

The **fourth stage** represents the application phase, in which a chosen optimization algorithm uses the trained surrogate model as its objective function to synthesize complete clustering pipelines for new datasets. While the first three stages correspond to the *learning phase* of PoAC (performed once to construct the meta-knowledge base), this final stage corresponds to the *optimization phase*, which is executed each time a new problem instance is addressed.

PoAC allows users to instantiate problem-specific configurations by selecting meta-features and CVIs that best express their clustering objectives (e.g., compactness, separability, robustness). This customization relies on domain understanding rather than automatic metric selection, ensuring flexibility while maintaining interpretability. Furthermore, PoAC is *algorithm-agnostic*, as it evaluates the quality of partitionings through CVIs rather than the algorithms that generate them. Any clustering algorithm, or combination thereof, can thus be incorporated into the search space without altering the surrogate model. Finally, PoAC is *decoupled from the optimization approach*: its surrogate model defines a learned objective function that can be optimized by any search strategy (e.g., evolutionary, Bayesian, or reinforcement-based). This separation allows the framework to adapt to different computational budgets or optimization paradigms while maintaining consistency in its learned knowledge base.

4.1 Problem Statement: Surrogate-Based PS for Clustering

We define a *dataset* $D = \{x^{(i)}\}_{i=1}^{i=n} \in \mathcal{P}(X)$ and X as a set of observations, each observation being $x^{(i)} \in X$, and n is the cardinality of D.

We define *meta-features* $\mu : \mathcal{P}(X) \to \mathbb{R}^l$ as a function that maps a dataset $D \in \mathcal{P}(X)$ to a point $\mu(D)$ of a l-dimensional numerical space.

We define a *preprocessing technique* $p : \mathcal{P}(X) \to \mathcal{P}(X')$ as a function that maps a dataset $D \in \mathcal{P}(X)$ to another dataset $p(D, \lambda_p) \in \mathcal{P}(X')$; we denote by $P_{X,X'}$ the set of preprocessing techniques compatible with X and X'; we denote $\lambda_p \in \Lambda_p$ as the hyperparameters associated with a given preprocessing method p.

We define a clustering *algorithm* as function $a : \mathcal{P}(X') \to \Pi(X')$ that takes a dataset $D \in \mathcal{P}(X')$ and returns a partition $a(D, \lambda_a) \in \Pi(X')$ of D, i.e., a set of subsets of D such that $\bigcup_{D' \in a(D,\lambda_a)} D' = D$ and $\forall D', D'' \in a(D, \lambda_a), D' \cap D'' = \emptyset$; we denote by $\lambda_a \in \Lambda_a$ the set hyperparameters associated with the selected algorithm.

We define a *CVI* $m : \Pi(X') \to \mathbb{R}$ as a function that measures the quality of a partition—for simplicity, we assume that for the values of m, the greater, the better; we denote by M_X the set of CVIs compatible with X.

We define a clustering *pipeline* over X a tuple $((p_1, \lambda_{p_1}), \ldots, (p_k, \lambda_{p_k}), (a, \lambda_a))$ such that $p_1 \in P_{X,X_1}$, $p_2 \in P_{X_1,X_2}$,..., $p_k \in P_{X_{k-1},X_k}$ is a sequence of mutually compatible preprocess-

ing techniques (withd their corresponding hyperparameters) and $a_1 \in A_{X_k}$ is a clustering algorithm (with its corresponding hyperparameters) compatible with the output of p_k, i.e., X_k. For the sake of clarity and without a practical loss of generality, provided that the used X is general enough, we consider, from now on, only clustering pipelines where all the elements (preprocessing techniques and clustering algorithm) operate on the same X. Under this assumption, and omitting the subscript for X, the set of all clustering pipelines defined over X is $P^* \times A \times \Lambda_{P^*} \times \Lambda_A$, where P^* denotes the set of sequences of elements of P. We denote by $(\vec{p}, a, \lambda)$ a pipeline for X, with $\vec{p} = (p_1, \ldots, p_k)$ is a (possibly empty) sequence of preprocessing techniques, a is the clustering algorithm, and $\lambda \in \Lambda = \Lambda_{P_*} \times \Lambda_A$ is the parametrization of both $\vec{p}$ and a.

Given a CVI $m \in M$ and a dataset $D \in \mathcal{P}(X)$, finding the best pipeline $(\vec{p}^\star, a^\star, \Lambda^\star)$ corresponds to solving the following optimization problem:

$$(\vec{p}^\star, a^\star, \Lambda^\star) = \underset{(\vec{p},a,\lambda)\in P^*\times A\times\Lambda}{\arg\max} (m \circ a \circ \vec{p})(D, \lambda)$$

where the operator $\circ$ denotes the composition of functions.

It can be seen that solving the optimization problem defined above by enumeration would require actually applying each candidate pipeline to the dataset D. We here propose to use, for each m, a *surrogate function* $f_m : P^* \times A \times \Lambda \times \mathbb{R}^l \to \mathbb{R}$ that, given a pipeline $(\vec{p}, a, \lambda)$ and the meta-features $\mu(D)$ a dataset D, approximates the value of the CVI m computed on the partition obtained applying the pipeline on the dataset, i.e.:

$$f_m((\vec{p}, a, \lambda), \mu(D)) \approxeq (m \circ a \circ \vec{p})(D, \lambda) \tag{1}$$

Solving the problem of finding the best pipeline for a CVI $m \in M$ and a dataset $D \in \mathcal{P}(X)$ can be hence cast to optimizing f_m:

$$\underset{(\vec{p},a,\lambda)\in P^*\times A\times\Lambda}{\arg\max} f_m((\vec{p}, a, \lambda), \mu(D)) \tag{2}$$

Intuitively, the pair m, f_m, that is, the CVI and the corresponding surrogate function, represents an intended-usage of the clustering. For instance, one m might be the Silhouette Index (SIL) for clustering aimed at visualization; another m might be Density-Based Cluster Validation (DBCV) index for anomaly detection. The surrogate functions capable of capturing the problem meta-features which are most useful in the two cases would be different: in particular, in this work we use different meta-features for the dataset.

5 PoAC for Visualization

In this work, we consider the visualization as the goal of a PoAC instance. Recently, the relevance of visualization for the interpretation of ML methods is increasing (Chatzimparmpas et al., 2020), specially in unsupervised learning, where it provides a unique lens through which complex data structures can be interpreted. The objective of clustering for visualization is to uncover meaningful patterns or groupings within datasets and represent

them in a visually comprehensible manner (Al-Jabery et al., 2019; Ezugwu et al., 2022). The aim is not only to identify clusters but also to convey their inherent relationships and structures graphically. This task finds applications across various domains, from exploratory data analysis to pattern recognition, where understanding the inherent organization of data fosters insights and informed decision-making (Luxburg et al., 2012). Effective visualization-driven clustering strategies contribute to the development of interpretable and actionable representations (Assent, 2012), facilitating a deeper understanding of intricate dataset structures, even in high-dimensional data (Strehl & Ghosh, 2003).

5.1 Problem Space Design

We have synthesized 6130 datasets to represent a vast range of clustering problems to compose the $\mathcal{P}(X)$. These datasets were created by combining different ranges of important clustering characteristics (Zellinger & Bühlmann, 2023), namely: number of dimensions, number of clusters, quantity of data points, imbalance ratio as well as the shapes of the clusters, described in Table 1.

To compose the observations X, we conducted a data augmentation process to generate a range of different partitionings of each dataset in $\mathcal{P}(X)$. This augmentation process consisted of inserting Gaussian noise (Lopes et al., 2019) one hundred times into the cluster labels of the $\mathcal{P}(X)$ datasets. The observations X are mapped in terms of μ, which consists of (i) the unsupervised meta-features extracted from the original datasets and CVIs calculated on the augmented partitioned data of $\mathcal{P}(X)$, and (ii) the external CVI (m), used as the target for the surrogate model.

5.2 Feature Space Mapping

This stage combines dataset descriptors with problem-aligned validation measures. We first describe the dataset meta-features and then the internal CVIs used to represent candidate partitionings.

5.2.1 Dataset Meta-Features

The meta-features selected for this work were based on the taxonomy proposed by Lorena et al. (2019), chosen for their demonstrated effectiveness and interpretability in characterizing data distributions in unsupervised learning settings (Rivolli et al., 2018, 2022; Vanschoren, 2018). In total, 37 meta-features were employed. While Table 2 summarizes the core feature

Table 1 Dataset synthesized for the PoAC's *problem space* described in Section 5

Feature	Range
Dimensions	2–100
Clusters	2–35
Samples	150–5000
Overlap	1e-6–1e-5
Aspect Ref	1.5–5
Aspect Max Min	1–5
Radius Max Min	1–5
Distributions	normal–exponential–gumbel
Imbalance Ratio	1–3

definitions, several statistical descriptors (e.g., mean, standard deviation) were computed for multiple meta-feature groups, such as `sd`, `var`, `mad`, `eigenvalues`, and `cohesiveness`, to better capture variability across attributes. This expansion from the base set to 37 features enhances the representational richness of the meta-space by incorporating measures of both central tendency and dispersion.

The selected descriptors collectively capture complementary aspects of the data, encompassing: (i) general structural properties (e.g., instance-to-attribute ratios), (ii) statistical moments and correlation structure, (iii) information-theoretic complexity, (iv) density and distance-based characteristics, and (v) landmarking measures related to clustering validation indices. This combination enables a comprehensive characterization of dataset behavior relevant to clustering and anomaly detection performance.

5.2.2 Internal CVI

We opted to use SIL (Rousseeuw, 1987) and Davies-Bouldin Score (DBS) (Davies & Bouldin, 1979) as CVIs. Their selection as visualization descriptors for clustering problems

Table 2 List of meta-features extracted from datasets to compose the surrogate model

Meta-Features	Group	Description
attr_to_inst	general	Ratio between the number of attributes.
inst_to_attr	general	Ratio between the number of instances and attributes.
nr_attr	general	Total number of attributes.
nr_inst	general	Number of instances (rows) in the dataset.
attr_conc	info-theory	Concentration coefficient of each pair of distinct attributes.
attr_ent	info-theory	Shannon's entropy for each predictive attribute.
wg_dist	concept	Weighted distance, that captures how dense or sparse is the example distribution.
cohesiveness	concept	Improved version of the weighted distance, that captures how dense or sparse is the example distribution.
one_itemset	itemset	One itemset meta-feature.
two_itemset	itemset	Two itemset meta-feature.
t2	complexity	Average number of features per dimension.
t3	complexity	Average number of PCA dimensions per points.
t4	complexity	Ratio of the PCA dimension to the original dimension.
cov	statistical	Absolute value of the covariance of distinct dataset attribute pairs.
eigenvalues	statistical	Eigenvalues of covariance matrix from dataset.
iq_range	statistical	Interquartile range (IQR) of each attribute.
mad	statistical	Median Absolute Deviation (MAD) adjusted by a factor.
median	statistical	Median value from each attribute.
nr_cor_attr	statistical	Number of distinct highly correlated pair of attributes.
sd	statistical	standard deviation of each attribute.
sparsity	statistical	Calculates the (possibly normalized) sparsity metric for each attribute.
t_mean	statistical	Trimmed mean of each attribute.
var	statistical	Variance of each attribute.
SIL	landmarker	Measures how well an object fits within its cluster (cohesion) versus its separation from other clusters.
DBS	landmarker	Evaluates clustering compactness and separation
ARI	landmarker	Quantifies clustering similarity by comparing the overlap between two clusterings while correcting for chance

is grounded in their ability to provide comprehensive insights into the quality and distinctiveness of clustering solutions. A high SIL suggests visually distinct and well-separated clusters, enhancing the interpretability of visual representations (Bagirov et al., 2023; Shahapure & Nicholas, 2020). Complementing this, a low DBS reinforces the notion of visually cohesive and distinguishable clusters, contributing to an enhanced visual understanding (Maulik & Bandyopadhyay, 2002; Thomas et al., 2013). The two CVIs assess complementary aspects in regards to visualization, their combination allows for a more reliable optimization function, i.e., providing more sound clustering solutions. We chose not to include other internal CVIs due to (i) not being complementary (i.e., measuring the same facets of the problem) and (ii) including many dimensions to optimize makes the optimization problem more challenging and resource costly.

SIL ranges from −1 to 1: a value close to 1 for a data point indicates that it is well-clustered, meaning it is further away from neighboring clusters than its own; a value around 0 suggests that the point lies on the boundary between two clusters; and a negative value indicates that the point may be assigned to the wrong cluster, as it is closer to a different cluster than its own. The average SIL across all points in a dataset provides an overall measure of clustering quality (Rousseeuw, 1987). Let the i, j be n-dimensional feature vectors (data points) and $i, j \in C_I$; C as a given cluster of the dataset D; $d(i, j)$ as the average Euclidean distance between data points i and j; $|C_I|$ as the number of data points in cluster C_I and $|N|$ is the total number of data points present in the dataset D. The Silhouete Score for can be calculated using Equation (3) if and only if $|C_I| > 1$.

$$\mathcal{SIL}(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \tag{3}$$

where, $a(i)$ is the average distance from the -th data point to the other data points in the same cluster, given by Equation (4).

$$a(i) = \frac{1}{|C_I| - 1} \sum_{j \in C_I, j \neq i} d(i, j) \tag{4}$$

And, $b(i)$ is the smallest average distance between the i-th data point of C_I and the k-th data point of C_K, where $C_I \neq C_K$, given by Equation (5).

$$b(i) = \min_{K \neq I} \frac{1}{|C_K|} \sum_{k \in C_K} d(i, k) \tag{5}$$

Finally, the overall SIL for the entire dataset D is the average of the SIL for all data points, as given by Equation (6):

$$\mathcal{SIL}_D = \frac{1}{|N|} \sum_{i=1}^{N} \mathcal{SIL}(i) \tag{6}$$

The DBS, as introduced by Davies and Bouldin (1979), is defined as the ratio of compactness within clusters and the separation between clusters. It ranges from 0 to infinite, where a

DBS closer to 0 indicates better (more compact) clustering, with well-defined and separated clusters. Let C_I be a cluster of a dataset D; σ the average Euclidean distance from each data point of C_I to its centroid A_I; $d(C_I, C_K)$ be the Euclidean distance between the A_I and A_K ; and the data-point $i \in C_I$; $|C|$ be the total number of clusters in D; and $|C_I|$ be the total number of data points in C_I.

Where, the compactness of C_I is given by Equation (7):

$$\sigma_I = \frac{1}{|C_I|} \sum_{i=1}^{|C_I|} ||i - A_I|| \tag{7}$$

The distance between the centroids is given by Equation (8):

$$d(C_I, C_K) = ||A_I - A_K|| \tag{8}$$

And, the DBS for the entire dataset D is calculated using the Equation (9):

$$\text{DBS}_D = \frac{1}{|C|} \sum_{I=1}^{|C|} \max_{K \neq I} \left(\frac{\sigma_I + \sigma_K}{d(C_I, C_K)} \right) \tag{9}$$

5.3 Surrogate Modeling

This stage first specifies the external target used to label the meta-instances and then describes the regression model that approximates that target.

5.3.1 External CVI

For m, we chose the Adjusted Random Index (ARI), as defined in Equation 4, to quantify the difference between the augmented partitions and the ground truth according to the original dataset. The intent for the composition of μ and m in this way is to create a range of possible partitionings that represents how visually different they are to the original clustering. By jointly employing these metrics with the meta-features, we aim to capture a nuanced understanding of the visual separability and coherence of clusters in our evaluation, ensuring a robust assessment of the visibility of clustering patterns in diverse datasets.

ARI is a essentially an expansion of the Rand Index (RI), which is a measure of similarity between two partitionings (Hubert & Arabie, 1985). It is an external CVI, meaning it is useful for evaluating the performance of clustering algorithms when the ground truth is known. The RI considers pairs of samples and classifies them as either concordant or discordant based on whether they are placed in the same or different clusters in both partitionings. It ranges from 0 to 1, where 1 indicates perfect agreement between the two partitionings, and 0 indicates no agreement beyond that expected by chance.

Let n be the number of elements in the set $D = \{o_1, \ldots, o_n\}$, and let $U = \{U_1, \ldots, U_I\}$ and $V = \{V_1, \ldots, V_J\}$ represent two different partitionings of D into I and J clusters, respectively.

The RI is calculated using the formula (10):

$$RI = \frac{\alpha + \beta}{\binom{n}{2}} \tag{10}$$

where:

- α is the number of pairs of elements (o_i, o_j) that are placed in the **same cluster** in both U and V.
- β is the number of pairs of elements (o_i, o_j) that are placed in **different clusters** in both U and V.

For all $1 \leq i, j \leq n, i \neq j$:

$$\alpha = \left|\{(o_i, o_j) \mid o_i, o_j \in U_k \text{ and } o_i, o_j \in V_l \text{ for some } k, l\}\right| \tag{11}$$

$$\beta = \left|\{(o_i, o_j) \mid o_i \in U_{k_1}, o_j \in U_{k_2} \text{ and } o_i \in V_{l_1}, o_j \in V_{l_2}, \text{ for } k_1 \neq k_2 \text{ and } l_1 \neq l_2\}\right| \tag{12}$$

In this sense, α counts the number of pairs in the same cluster in both partitionings, while β counts pairs in different clusters in both partitionings. The denominator $\binom{n}{2}$ represents the total number of possible pairs of elements in D.

The ARI accounts for chance agreement, providing a normalized measure that ranges from −1 to 1. A score of 1 indicates perfect agreement, 0 suggests agreement expected by chance, and negative values imply worse-than-chance agreement. The formula for the ARI is given by:

$$\text{ARI} = \frac{\sum_{ij}\binom{n_{ij}}{2} - \left[\sum_i \binom{\alpha_i}{2} \sum_j \binom{\beta_j}{2}\right] \Big/ \binom{n}{2}}{\frac{1}{2}\left[\sum_i \binom{\alpha_i}{2} + \sum_j \binom{\beta_j}{2}\right] - \left[\sum_i \binom{\alpha_i}{2} \sum_j \binom{\beta_j}{2}\right] \Big/ \binom{n}{2}} \tag{13}$$

This formula normalizes the RI by accounting for random chance, making the ARI a valuable tool for assessing clustering accuracy in various fields, including biology, image analysis, and social sciences.

5.3.2 Surrogate Model

To select an appropriate surrogate model f_m, we evaluated several regressors, including Random Forest, XGBoost, Gradient Boosting, MLP, SVR, Ridge, Lasso, and Gaussian Process Regressor (GPR). Using a 10-fold cross-validation to train different models on the visualization meta-data, Random Forest and XGBoost achieved the highest predictive performances with low RMSE and MAE, while the other models performed noticeably worse (see Table 3). Considering both predictive accuracy and interpretability, we selected Random Forest, which captures complex relationships while providing feature importance

Table 3 Comparison of surrogate model regressors on the visualization meta-dataset

Regressor	R^2 mean	RMSE mean	MAE mean
RandomForest	0.909	0.069	0.052
XGBoost	0.910	0.069	0.051
GradientBoosting	0.855	0.087	0.067
MLP	0.866	0.084	0.062
SVR	0.288	0.193	0.155
Ridge	0.489	0.163	0.131
Lasso	0.195	0.205	0.167
GPR	0.039	0.224	0.185

Metrics are averaged over 10-fold cross-validation, sorted by RMSE mean

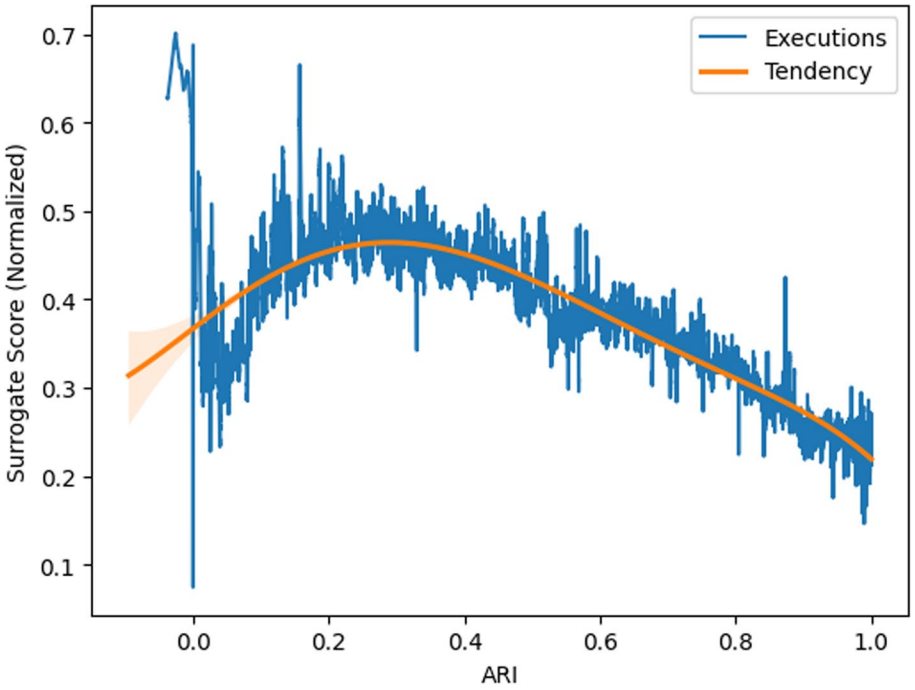


Fig. 2 Correlation between predicted and true ARI values obtained from the PoAC surrogate model for visualization tasks. Each point represents a dataset instance. The strong alignment ($R^2 = 0.91$) confirms that the Random Forest surrogate accurately predicts clustering quality based on meta-features and CVIs

scores that help understand the influence of individual meta-features and clustering validation indices.

The Random Forest regressor was trained using the observations X (meta-features and clustering validation indices) as independent variables and the ARI as the dependent variable. The model provides a reliable fitness function for optimization by effectively capturing the relationships between dataset characteristics and clustering performance. Training was performed with 10-fold cross-validation, yielding strong predictive performance ($R^2 \sim 0.91$, RMSE ~ 0.069, MAE ~ 0.052), indicating neither overfitting nor underfitting (see Figure 2).

Feature importance analysis of the trained Random Forest, shown in Figure 3, identifies SIL (0.467) and DBS (0.226) as the dominant predictors. The compact figure reports the ten highest-ranked features; every omitted individual feature has importance at most 0.013.

The remaining top-ranked descriptors capture entropy, sparsity, covariance, variance, eigenvalue structure, within-group distance, and interquartile range. Their smaller individual importances indicate that the surrogate relies primarily on the two CVIs while retaining complementary information about the dataset.

5.4 Function Optimization

We extended the capabilities of the AutoML framework Tree-based Pipeline Optimization Tool (TPOT), presented by Olson and Moore (2016), to allow it to synthesize pipelines for clustering problems. This extension consists of a set of important modifications. Mainly, the incorporation of the surrogate model to serve as a fitness function for the evolutionary optimization process. This entails the extraction of meta-features on new unseen clustering datasets, and the measuring of both SIL and DBS for each synthesized clustering pipeline solution, as displayed in Figure 4. TPOT was chosen due to its robustness, high level of maintenance, and widespread use within the machine learning community, making it a reliable and well-supported tool for extending into new domains. These qualities make TPOT particularly suitable for our purposes, as it provides a stable and flexible framework for automating complex machine learning tasks.

Furthermore, we have replaced the conventional TPOT's classification operators with clustering counterparts, as displayed in Table 4. Through these modifications, we effectively extended TPOT's utility to the realm of clustering problems, providing a versatile and adaptive AutoML tool[1].

[1] We created a code repository for this version of TPOT, called Tpot Clustering.

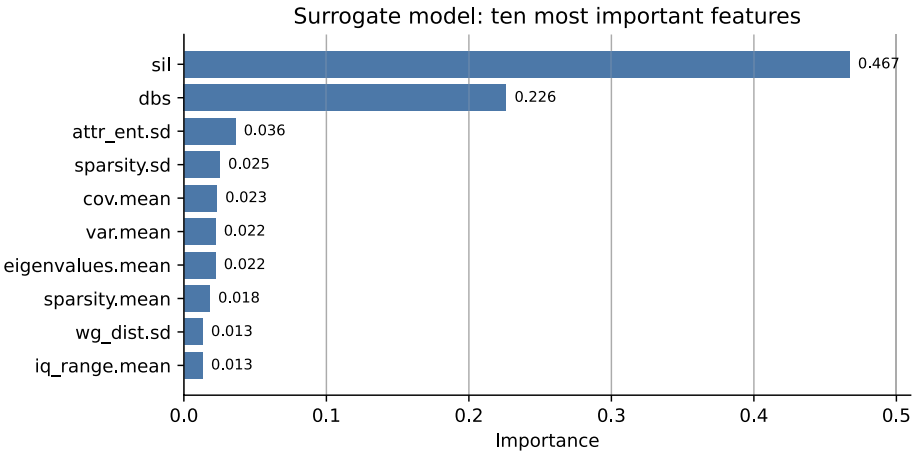


Fig. 3 Top-ten feature importances of the Random Forest surrogate trained on the visualization meta-dataset. SIL and DBS dominate the ranking; importances are averaged across the ensemble's trees

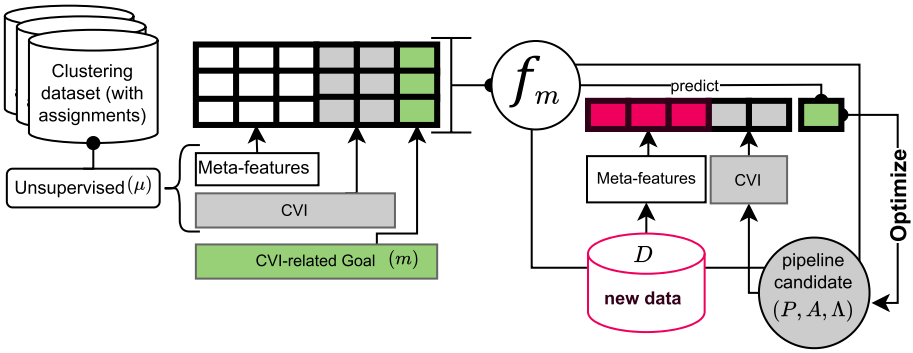


Fig. 4 Pipeline optimization process for clustering problems within the PoAC framework. The process begins by extracting meta-features (μ) and CVI from clustering datasets. This CVI-related goal (m) serves as the target for the surrogate model f_m, which is used to predict the quality for pipeline candidates (P, A, Λ) on new, unseen data (D). The surrogate model then optimizes the pipeline, evaluating potential solutions based on their predicted (m), using extracted meta-features from the new data and CVI from pipelines to inform the optimization process

Table 4 Clustering operators included in the TPOT configuration

Operator	Type	Hyperparameters
Agglomerative Clustering	cluster	•n_clusters: range(2, 23) •metric: euclidean •linkage: ward
DBSCAN	cluster	•eps: range(2, 23) •min_samples: [1e-3, 1e-2, 1e-1, 1., 10., 100.] • metric: [10, 25, 50] • leaf_size: [3, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50]
KMeans	cluster	• n_clusters: range(2, 23) • init: [k-means++, random]
Mini Batch KMeans	cluster	• n_clusters: range(2, 23) • eps: range(2, 23)
Spectral Clustering	cluster	• n_clusters: range(2, 23) • eigen_solver: [arpack, lobpcg, amg] • affinity: [nearest_neighbors, rbf, precomputed, precomputed_nearest_neighbors]
MinMax Scaler	preprocessing	-
Normalizer	preprocessing	• norm: [l1, l2]
Standard Scaler	preprocessing	-
Variance Threshold	feature selection	• threshold: [0.1, 0.25]
PCA	decomposition	• n_components: [2, 3, 5, 10]
Fast ICA	decomposition	• n_components: [2, 3, 5, 10]

5.5 Baselines

We designed and performed experiments to evaluate the quality of PoAC when compared to the reproducible state-of-the-art frameworks that focus on CASH, namely *ML2DAC* from Treder-Tschechlov et al. (2023), *Autocluster* proposed by Liu et al. (2021), *cSmartML* as presented in ElShawi et al. (2021) and, finally, *AutoML4Clust* presented in Tschechlov et al. (2021). As for PoAC, we specifically created an instance of the PoAC framework for visualization as a clustering problem (Ezugwu et al., 2022) and assessed the performance of pipeline optimizations. In the context of the proposed *problem-oriented* methodology, prioritizing visualization as a clustering problem underscores its significance as a user-defined goal, catering to scenarios where the interpretability and visual clarity of clustering solutions are paramount.

The datasets employed in this evaluation are not restricted to visualization-oriented contexts. Both the synthetic and real-world datasets encompass a wide range of data distributions, dimensionalities, and cluster structures representative of general clustering problems. In this setup, PoAC's *visualization* instance does not rely on visual data but defines an interpretable clustering objective that favors partitions aligned with human-understandable structure and separability. Consequently, the results reflect general unsupervised clustering performance rather than a visualization-specific data domain.

We evaluated the frameworks on two complementary groups. The first comprises 100 synthetic clustering datasets from the benchmark of Silva et al. (2024), spanning the ranges in Table 5. The second comprises 22 UCI datasets (Kelly & Longjohn, 2017) used in prior evaluations of ML2DAC and AutoCluster (Table 5).

Together, the synthetic and real-world groups test PoAC across varied dimensionalities, sample sizes, cluster structures, and application domains.

Table 5 Validation data used in the experiments: (a) ranges covered by the 100 synthetic datasets and (b) characteristics of the 22 real-world UCI datasets

(a) Synthetic datasets

Feature	Range	Feature	Range
Dimensions	2–100	Clusters	2–35
Samples	150–5,000	Overlap	10^{-6}–10^{-5}
Aspect Ref	1–10	Aspect Max Min	1–10
Radius Max Min	1–10	Imbalance Ratio	1–3

(b) Real-world datasets

Dataset	n	d	k	Dataset	n	d	k
arrhythmia	452	262	13	segment	2,310	19	7
balance-scale	625	4	3	sonar	208	60	2
dermatology	366	34	6	tae	151	5	3
ecoli	336	7	8	thy	215	5	3
german	1,000	7	2	vehicle	846	18	4
glass	214	9	6	vowel	990	10	11
haberman	306	3	2	wdbc	569	30	2
heart-statlog	270	13	2	wine	178	13	3
iono	351	34	2	wisc	699	9	2
iris	150	4	3	yeast	1,484	8	10
letter	20,000	16	26	zoo	101	16	7

Table 6 Frameworks performance regarding ARI, SIL, and DBS for the validation datasets (mean, median, and standard deviation).

Framework	ARI			SIL			DBS		
	Mean	Median	SD	Mean	Median	SD	Mean	Median	SD
PoAC	**0.70**	**0.87**	**0.27**	**0.63**	**0.63**	**0.07**	**0.52**	**0.52**	**0.18**
ML2DAC	0.68	0.83	0.32	0.40	0.41	0.19	1.68	1.34	1.31
AutoML4Clust	0.59	0.64	0.29	0.37	0.35	0.16	1.39	1.26	0.76
Autocluster	0.59	0.61	0.22	0.23	0.19	0.19	2.43	2.11	1.63
cSmartML	0.39	0.39	0.29	0.25	0.25	0.20	1.37	1.28	0.60

The highest value for eachmetric was highlighted in bold

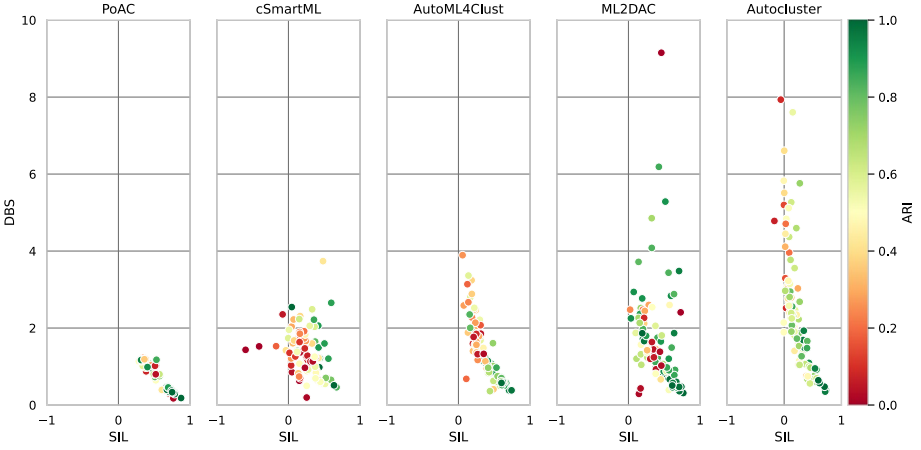


Fig. 5 Comparison of clustering quality across frameworks (PoAC, ML2DAC, AutoML4Clust, Autocluster, cSmartML) on validation datasets. Each subplot shows the relationship between SIL and DBS, with color intensity indicating ARI. PoAC achieves clusters concentrated in the optimal region (high SIL, low DBS), indicating better separation and compactness compared to competing AutoML methods

5.6 Results and Discussion

We applied each one of the baseline frameworks, as well as the proposed PoAC, on the group of one hundred validation datasets. The results are presented in Table 6. The obtained results pertaining ARI will be discussed in subsubsection 5.6.1.

As reported in Table 6, PoAC obtained mean ARI, SIL, and DBS values of 0.70, 0.63, and 0.52, respectively. The corresponding means were 0.68, 0.40, and 1.68 for ML2DAC; 0.59, 0.37, and 1.39 for AutoML4Clust; 0.59, 0.23, and 2.43 for Autocluster; and 0.39, 0.25, and 1.37 for cSmartML. Thus, among the evaluated frameworks, PoAC combined the highest mean ARI and SIL with the lowest mean DBS (lower DBS is better).

These results indicate that PoAC produced partitions that were both well separated according to the internal indices and more consistent with the ground-truth labels. The claim is limited to the evaluated datasets, search spaces, and computational settings.

As illustrated in Figure 5, PoAC's results are concentrated toward the favorable region of high SIL and low DBS. Their ARI coloring also indicates greater agreement with the ground truth on average. The competing methods are more dispersed in the SIL–DBS plane. This

figure is descriptive rather than inferential; the statistical comparison of ARI is reported next.

5.6.1 Clustering Performance Analysis

To evaluate agreement between the clustering solutions and the original labels of the validation datasets, we compared their ARI distributions. PoAC achieved the highest mean ARI (0.70), followed by ML2DAC (0.68), AutoML4Clust (0.59), Autocluster (0.59), and cSmartML (0.39), using the rounded values in Table 6.

A statistical comparison was performed over five populations, each consisting of 100 paired ARI samples. Preliminary tests indicated that some populations (AutoML4Clust, Autocluster, and PoAC) deviated from normality. Consequently, we applied the non-parametric Friedman test with a 95% confidence level, which confirmed significant differences among the medians of the populations. A post-hoc Nemenyi test (Nemenyi, 1963), with a critical distance (CD) of 0.61, was then used to identify pairwise differences. As shown in Figure 6, no significant differences were found within the following groups: (i) AutoML-4Clust, Autocluster, and ML2DAC; and (ii) ML2DAC and PoAC. All other comparisons revealed significant differences, particularly between cSmartML and any other framework, and between PoAC and both AutoML4Clust and Autocluster.

Figure 6 summarizes these results by ranking the frameworks according to their mean ranks (lower is better). The horizontal bars indicate groups of methods whose performances are not significantly different at the 95% confidence level. The proximity between PoAC and ML2DAC confirms that both achieve similarly high ARI values, whereas the remaining frameworks form a distinctly lower-performing group. Therefore, while ML2DAC performs competitively, PoAC attains the lowest mean rank (2.22) and stands at the upper boundary of the top statistical group, consolidating its position as the leading framework across the evaluated datasets.

To ensure a fair and directly comparable evaluation between the two top-performing frameworks, PoAC and ML2DAC, we replicated the real-world experiment from the ML2DAC study using the same set of UCI datasets (referred to as the second validation group). Because ML2DAC does not include automated preprocessing, we conducted an ablation study of PoAC by removing all preprocessing operators from its search space. Under these matched conditions, PoAC achieved a mean ARI of 0.26 (median = 0.18, SD = 0.28), closely aligning with ML2DAC's reported mean ARI of 0.27 for its full configuration and 0.19 for its simplified variant (Treder-Tschechlov et al., 2023). These results

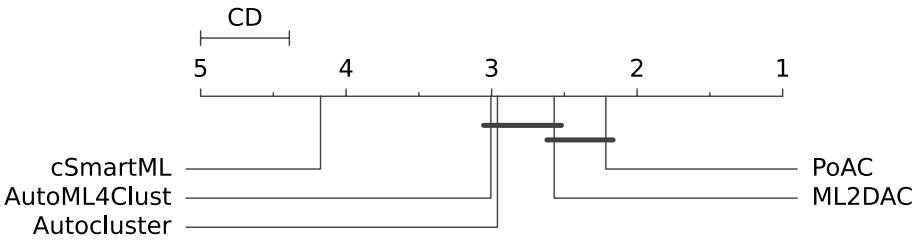


Fig. 6 Frameworks ranked by mean rank according to the Nemenyi test on ARI (lower values indicate better performance). From last to first: cSmartML (MR=4.18), AutoML4Clust (MR=3.01), Autocluster (MR=2.96), ML2DAC (MR=2.57), and PoAC (MR=2.22)

demonstrate that even without preprocessing, PoAC achieves comparable performance to ML2DAC while retaining the benefits of a meta-learned, transferable optimization strategy.

Overall, PoAC obtained the strongest aggregate results in the first experiment. On the UCI benchmark, its configuration without preprocessing achieved a mean ARI comparable to the reported ML2DAC configurations. These results support the use of the learned surrogate as a search objective under the evaluated conditions, while broader generalization remains subject to the limitations in section 7.

5.6.2 Runtime Analysis and Computational Cost

Constructing the meta-space constitutes the primary computational cost in both PoAC and ML2DAC, as it involves extracting descriptive meta-features from large numbers of datasets. For PoAC, computing around 40 meta-features and 3 CVIs across 6,000 synthetic datasets required approximately 4 hours and 52 minutes on a workstation with an Intel i9 CPU and 64 GB of RAM, corresponding to an average of 2.9 seconds per dataset.

In comparison, ML2DAC (Treder-Tschechlov et al., 2023) reports per-dataset meta-feature extraction times ranging from 0.1 to 82.8 seconds, depending on the selected meta-feature set. Its most comprehensive configuration (`MF_AutoCluster`) averages 35.7 seconds per dataset, while PoAC provides competitive meta-feature coverage with substantially reduced extraction time, facilitating large-scale meta-space construction.

5.6.2.1 Optimization and Inference Time ML2DAC constructs its meta-space from a predefined set of 78 datasets, each optimized independently with a two-hour time budget, totaling approximately 156 hours (Treder-Tschechlov et al., 2023). Once this meta-space is built, inference on new datasets takes on average 74 seconds per dataset when using the full meta-feature configuration, enabling rapid deployment on unseen problems.

For PoAC, meta-feature extraction and meta-space construction required less than five hours. Under the original setup, full optimization across both sets of benchmark datasets required approximately 306.7 hours, corresponding to 100 generations with a population of 100 individuals per dataset and no early stopping. To align with ML2DAC's experimental protocol, we also evaluated PoAC under a two-hour time budget per dataset, resulting in a total optimization time of 44 hours for the 22 datasets in the benchmark.

Once the PoAC surrogate is trained, inference on new datasets is efficient and fully configurable, depending on user-defined parameters such as population size, number of generations, or optional time budgets. Overall, PoAC achieves a favorable balance between meta-space construction and optimization, offering competitive computational efficiency and strong scalability across diverse downstream tasks.

5.6.3 Generated Pipeline Complexity

An analysis of the mean pipeline complexity (i.e., the number of pipeline steps) across generations reveals a clear trend within the PoAC framework. As shown in Figure 7, PoAC consistently evolves towards pipelines with a mean complexity of at least two steps. This stabilization suggests an optimal balance between essential preprocessing operations and the clustering algorithm itself. The trend highlights PoAC's ability to autonomously con-

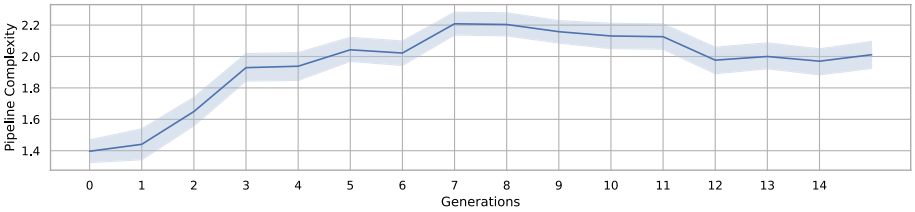


Fig. 7 Evolution of pipeline complexity during the PoAC optimization process. Complexity is measured as the average number of components (preprocessing + clustering steps) per pipeline generation. The model converges toward compact pipelines with approximately two steps, suggesting an optimal trade-off between performance and simplicity

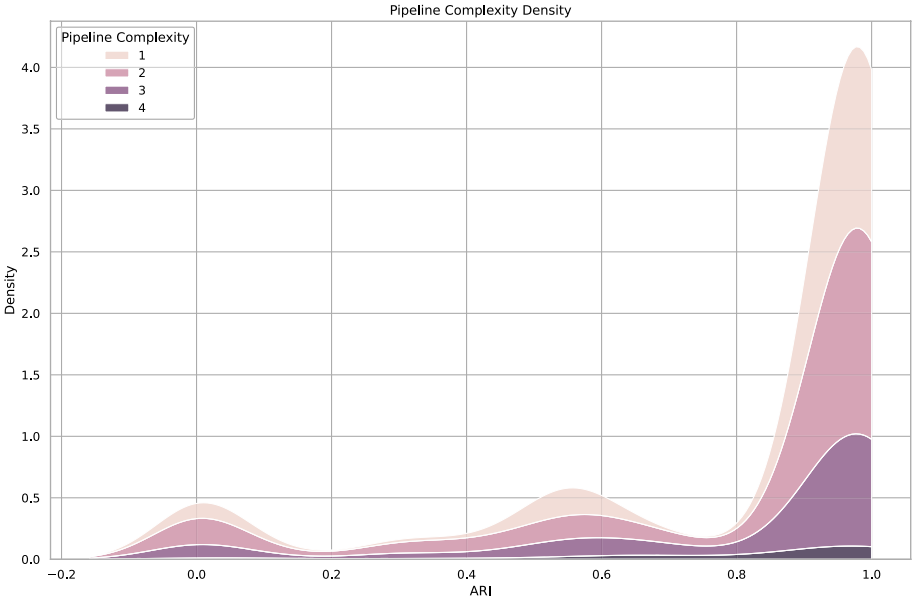


Fig. 8 Relationship between pipeline complexity and ARI performance across generations. Higher-performing pipelines (with higher ARI) tend to be simpler, typically involving one or two processing steps. This trend reflects PoAC's capacity to discover minimal yet effective clustering pipelines while preserving adaptability for complex datasets

verge toward compact yet effective configurations, minimizing unnecessary complexity while maintaining strong clustering performance.

As depicted in Figure 8, the distribution of pipeline complexities reveals a clear association with ARI values. Pipelines yielding lower ARI scores display a broader spread of complexities, reflecting an exploratory search behavior. In contrast, higher-performing pipelines (with higher ARI) are concentrated around one or two steps, consistent with the evolutionary convergence observed earlier. Nonetheless, a small number of high-performing pipelines with three or four steps remain, indicating that PoAC retains flexibility to adapt its structure when additional preprocessing improves performance. Overall, the framework tends to favor simpler, well-balanced pipelines that maintain high effectiveness.

The influence of pipeline composition on clustering quality is illustrated in Figure 9 using the UCI *dermatology* dataset (Ilter & Guvenir, 1998). In this example, ML2DAC recommends a simple CASH-based solution, *KMeans* with two clusters, whereas PoAC proposes a multi-step pipeline including (i) *MinMaxScaler* for normalization and (ii) *MiniBatchKMeans* as the clustering algorithm, with batch size 10 and six clusters. The resulting pipeline produces a clustering more faithful to the ground truth, demonstrating the advantage of solving the full pipeline search (PS) problem instead of limiting the optimization to algorithm and hyperparameter selection (CASH). This example illustrates how PoAC leverages preprocessing choices to enhance clustering outcomes.

An important factor underlying this flexibility is PoAC's algorithm-agnostic design, which directly relates to the diversity of effective pipeline configurations reported above. Unlike other AutoML approaches that build a meta-database tied to specific algorithms and hyperparameter settings, PoAC introduces a label-augmentation process during meta-learning, adding controlled noise to the synthetic datasets' cluster labels. This allows the surrogate model to learn algorithm-independent partitioning patterns, enabling it to generalize across a broad range of clustering strategies. Consequently, PoAC can recommend different, but equivalently performant, pipeline configurations without retraining the surrogate model when the search space changes. This capability explains the observed adaptability in pipeline composition and reinforces PoAC's robustness as a truly algorithm-agnostic AutoML framework.

5.6.4 Ablation Study

We conducted an ablation study to isolate the contributions of dataset meta-features and the surrogate objective. The complete PoAC configuration uses meta-features and the two CVIs (SIL and DBS) as surrogate inputs. We compare it with three variants:

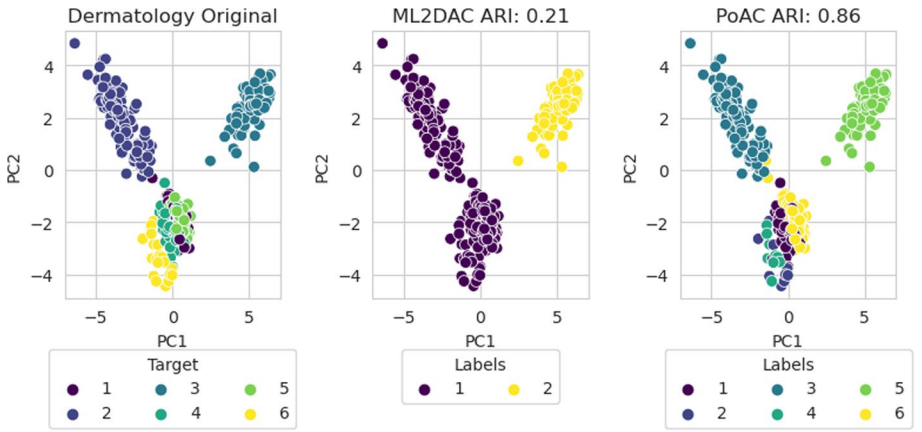


Fig. 9 Example of clustering pipeline recommendations for the Dermatology dataset. PoAC proposes a two-step pipeline combining MinMaxScaler normalization and MiniBatchKMeans, achieving a clustering more faithful to the ground truth than ML2DAC's single-step KMeans solution. This demonstrates PoAC's advantage in full pipeline synthesis and problem-oriented optimization

1. PoAC CVI: a Random Forest surrogate trained with SIL and DBS, but without dataset meta-features;
2. PoAC SIL: direct optimization of SIL only; and
3. PoAC DBS: direct minimization of DBS only.

The analysis uses the same 100 synthetic validation datasets for all four strategies. Each strategy was executed ten times per dataset. For each run, the best candidate pipeline according to the corresponding optimization objective was retained. Metric values were then averaged across the ten runs to obtain one dataset-level outcome per strategy. The reported mean, median, and standard deviation were then calculated across the 100 dataset-level outcomes, so that every dataset received equal weight. ARI, which uses the known synthetic labels only for evaluation, measures how well each selected partition recovers the underlying structure.

Figure 10 shows that complete PoAC achieves the highest mean and median ARI, 0.71 and 0.75, respectively. PoAC CVI and PoAC SIL have similar central performance: their mean/median ARI values are 0.52/0.55 and 0.53/0.53, respectively, whereas PoAC DBS obtains 0.42/0.27. The ARI standard deviations are 0.28, 0.20, 0.41, and 0.41 for PoAC, PoAC CVI, PoAC SIL, and PoAC DBS, respectively. Thus, PoAC CVI has the smallest dispersion, but around a lower ARI level than complete PoAC, while both single-CVI strategies are substantially more variable.

The dataset-level results also show that all strategies obtain lower mean ARI as dimensionality and the number of clusters increase, but the degradation is smaller for complete PoAC. For datasets with fewer than 50 dimensions ($n = 64$), its mean ARI is 0.77, compared with 0.63 for PoAC CVI, 0.71 for PoAC SIL, and 0.54 for PoAC DBS. At 50 or

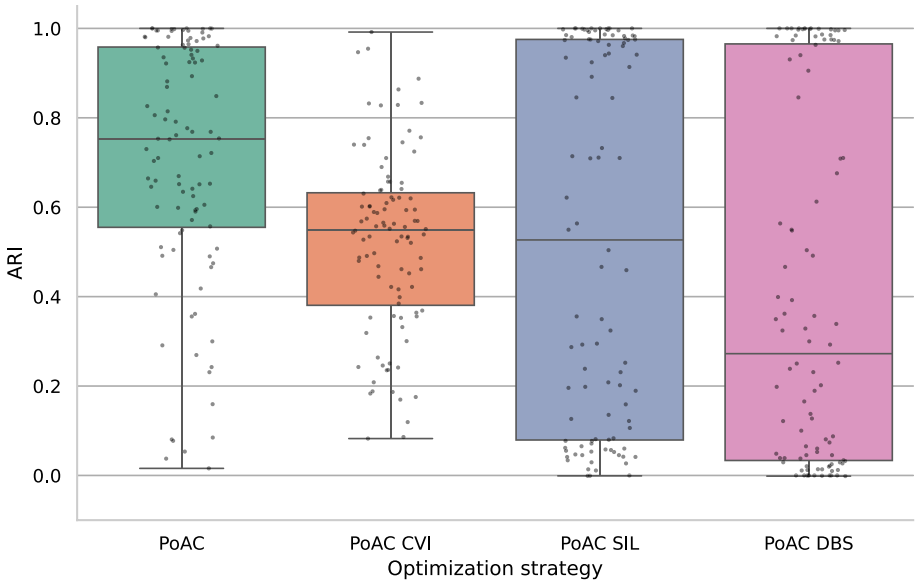


Fig. 10 ARI distributions across the 100 synthetic validation datasets for complete PoAC and the three ablated strategies. Complete PoAC obtains the highest mean and median ARI. PoAC CVI is more concentrated around a lower ARI level, whereas the single-CVI strategies exhibit greater dispersion

more dimensions ($n = 36$), the respective means are 0.59, 0.34, 0.22, and 0.20. Similarly, for fewer than 10 clusters ($n = 54$), the means are 0.78, 0.62, 0.77, and 0.64; for 10 or more clusters ($n = 46$), they fall to 0.62, 0.41, 0.25, and 0.16.

Table 7 completes the analysis by relating the optimized internal indices to external clustering quality. As expected, PoAC SIL achieves the highest mean SIL (0.76) and PoAC DBS the lowest mean DBS (0.28), but their mean ARI values remain below that of complete PoAC. PoAC CVI also underperforms complete PoAC and does not improve upon SIL-only optimization in mean ARI despite combining both internal indices in a surrogate. Because complete PoAC differs from PoAC CVI by conditioning the surrogate on dataset meta-features, this comparison supports the value of the meta-features for adapting the objective to the problem at hand. More generally, the results show that optimizing an internal index, either directly or through a CVI-only surrogate, does not necessarily recover the ground-truth partition.

6 PoAC for Anomaly Detection

This section applies PoAC to anomaly detection tasks using the methodology introduced in section 5. The configuration comprises four problem-specific components: (i) synthetic datasets with controlled anomaly levels, (ii) meta-features and CVIs tailored to anomaly detection, (iii) a surrogate trained on the resulting meta-dataset, and (iv) optimization of anomaly-detection pipelines using that surrogate as a computationally inexpensive objective.

6.1 Problem Space Design

In anomaly detection, particularly within unsupervised settings, the challenge lies in identifying data points that deviate significantly from the majority without prior labeling. This approach is prevalent due to the scarcity of labeled anomaly data in many real-world applications (Chandola et al., 2009).

To model such scenarios, we focus on *global anomalies*, also known as point anomalies (Chandola et al., 2009), which are data points that lie far outside the support of the original distribution. These are the most common targets in unsupervised anomaly detection benchmarks (Mejri et al., 2024). In contrast, *local anomalies* refer to points that deviate from their immediate neighborhood but not globally, and are often more challenging to detect without domain-specific knowledge (Agyemang, 2024).

Table 7 Performance of the optimization strategies on the synthetic validation datasets.

Strategy	ARI			SIL			DBS		
	Mean	Median	SD	Mean	Median	SD	Mean	Median	SD
PoAC	**0.71**	**0.75**	0.28	0.54	0.51	0.17	0.76	0.80	0.37
PoAC CVI	0.52	0.55	0.20	0.46	0.46	0.03	0.80	0.81	0.10
PoAC SIL	0.53	0.53	0.41	**0.76**	**0.75**	0.09	0.36	0.34	0.31
PoAC DBS	0.42	0.27	0.41	0.69	0.70	0.20	**0.28**	**0.30**	0.13

Values are the mean, median, and standard deviation (SD) across the 100 dataset-level outcomes. For each dataset and strategy, metric values are first averaged across the ten retained run-level pipelines. Lower DBS is better

The highest value for each metric was highlighted in bold

As in the visualization case, the problem space for anomaly detection is constructed by generating a large and diverse collection of synthetic datasets. We use the `repliclust` library to synthesize multi-cluster distributions with varying dimensionality, density, overlap, and imbalance. To model anomaly detection scenarios and mimic the challenges of uncurated real-world datasets (Aqeel et al., 2025), we inject anomalies at three controlled contamination rates $1\%, 3\%, 5\%$. This three-level approach simulates varying degrees of anomaly prevalence, allowing the meta-space to capture both subtle and extreme deviations from normality.

The resulting problem space comprises 4,600 synthetic mixture-model data profiles generated with `repliclust` and labeled with ground-truth anomaly indicators. These profiles form the basis for extracting meta-features, computing internal validity indices, and training the surrogate model for anomaly-detection pipeline optimization.

6.2 Feature Space Mapping

Each dataset in the problem space is mapped into a vector of descriptive statistics to enable transfer of knowledge across tasks. For anomaly detection, we adopt a tailored set of 20 meta-features extracted with `pymfe`, focusing on properties that are particularly relevant for distinguishing anomalies from normal structure. These include: measures of distributional spread and dispersion (`iq_range.sd`, `sd.mean`, `var.mean`, `kurtosis.mean`), density- and distance-based descriptors (`wg_dist.mean`, `wg_dist.sd`, `cohesiveness.mean`), dimensionality and attribute ratios (`inst_to_attr`, `attr_to_inst`, `nr_attr`, `nr_inst`, `nr_cor_attr`), as well as structural statistics such as eigenvalue spectra (`eigenvalues.mean`, `eigenvalues.sd`, `cov.mean`, `cov.sd`), correlation measures (`cor.mean`, `cor.sd`), and pattern-based descriptors (`t2`, `t3`, `t4`, `one_itemset.mean`, `two_itemset.mean`, `attr_ent.mean`, `attr_conc.mean`). Together, these features capture density, variance structure, redundancy, and higher-order interactions that influence anomaly detectability.

In addition to meta-features, we also compute CVIs. We use SIL to measure cluster cohesion and separation. Additionally, the Calinski–Harabasz Score (CHS) (Caliński & Harabasz, 1974) quantifies the ratio of between-cluster variance to within-cluster variance, rewarding compact and well-separated clusters. Finally, the DBCV (Moulavi et al., 2014) evaluates the relative density connectedness within clusters against density separability between clusters, yielding values in $[-1, 1]$, where values closer to 1 indicate well-formed, density-consistent clusters and negative values suggest poor structure or excessive noise, which may reflect the presence of global anomalies (Chicco et al., 2025; Ester et al., 1996).

As in the visualization setup, ARI serves as the external target, treating anomalies as a separate class. Each data profile gives rise to multiple partition–CVI realizations, resulting in approximately 40,000 meta-instances described by around 30 meta-feature and CVI attributes.

6.3 Surrogate Modeling

We train a surrogate model that predicts clustering performance given a dataset's meta-features and internal CVIs. For this purpose, we also employ a Random Forest Regressor (Breiman, 2001).

The surrogate is trained on these approximately 40,000 meta-instances, with ARI relative to the ground-truth anomaly labels as the target. Cross-validation yielded $R^2 \approx 0.81$, RMSE ≈ 0.095, and MAE ≈ 0.071, indicating that the model captures a substantial part of the relationship between dataset characteristics and the ARI achieved by candidate partitions.

Feature importance analysis indicates that both meta-features and CVIs contribute meaningfully to the surrogate's predictions. The most influential descriptors include SIL, number of attributes, DBCV, within-group distance (wg_dist.sd), correlation measures (cor.mean, cor.sd), kurtosis, CHS, interquartile range (iq_range.sd), and attribute concentration (attr_conc.mean). The top ten features by importance are: SIL (0.3169), nr_attr (0.1237), DBCV (0.0912), wg_dist.sd (0.0641), cor.mean (0.0544), cor.sd (0.0388), kurtosis.mean (0.0358), CHS (0.0319), iq_range.sd (0.0314), and attr_conc.mean (0.0280).

Once trained, the surrogate serves as an efficient proxy for pipeline evaluation, guiding the optimizer toward pipelines predicted to yield high-quality anomaly-detection results.

6.4 Function Optimization

The optimization stage follows the same pipeline search methodology described in Section 5, but adapted to the anomaly detection setting. We employ `TPOTClustering` as the optimizer, using the surrogate model as the scoring function to efficiently guide the search toward promising pipelines. The surrogate integrates both meta-features and CVIs, as described earlier, and predicts the expected ARI of candidate anomaly detection pipelines.

The search space is designed to include density-based clustering methods commonly used for anomaly detection, as well as standard preprocessing and dimensionality-reduction operators. Table 8 summarises the search space explored in our experiments.

In practice, the optimizer runs for 10 generations with a population size of 30, balancing exploration of the pipeline space with computational efficiency.

For the baseline configuration, we used the `IsolationForest` implementation from `scikit-learn` with the following parameters: `n_estimators=100`, `max_samples='auto'`, `max_features=1.0`, and `contamination='auto'`. We deliberately retained the default unsupervised setting (`contamination='auto'`) to ensure methodological parity with PoAC, which likewise receives no prior information about the proportion of anomalies. This configuration allows both approaches to operate under the same information constraints, enabling a fair comparison of their intrinsic anomaly detection capabilities.

PoAC and IForest were evaluated under identical preprocessing and data-splitting conditions, using the same injected contamination levels (1%, 3%, and 5%) and evaluation metrics (precision, recall, F1, and ROC AUC). Each experiment was repeated five times with

Table 8 Search space for anomaly-detection pipelines in PoAC

Operator	Hyperparameters
`DBSCAN`	`eps`$\in \{0.1, 0.5, 1.0, 5.0\}$; `min_samples`$\in \{5, 10, 20\}$; metric = `euclidean`
`OPTICS`	`min_samples`$\in \{5, 10, 20\}$; $\xi \in \{0.01, 0.05, 0.1\}$; `min_cluster_size`$\in \{0.05, 0.1, 0.2\}$
`StandardScaler`	–
`MinMaxScaler`	–
`PCA`	`n_components`$\in \{2, 3, 5\}$

different random seeds. Precision, recall, and F1 were computed separately in each run and then averaged; consequently, the displayed mean F1 is not necessarily the harmonic mean of the displayed mean precision and recall.

6.5 Results and Discussion

On the task of anomaly detection, we have evaluated PoAC against the classical IForest across eight representative datasets, chosen to capture diverse domains and anomaly characteristics. The datasets include medical and health records (*haberman, heart-statlog*), industrial monitoring (*vehicle*), image or segment recognition tasks (*segment, dermatology*), and classical benchmarks (*balance-scale, wdbc, yeast*). Figure 11 presents the comparative F1 scores and ROC AUC for each dataset.

Across these datasets, PoAC frequently achieves higher F1 scores than IForest, notably on *balance-scale, haberman, segment*, and *yeast*. This indicates that PoAC can more precisely identify anomalies in contexts where both missed anomalies and false alarms carry significant consequences, such as in medical diagnostics or industrial quality control. In contrast, ROC AUC values are broadly similar between the two methods, suggesting comparable ability to distinguish anomalous from normal samples. PoAC's F1 advantage is most relevant when exact detection is prioritized over overall separability (Table 9).

Analyzing precision and recall explains why PoAC achieves higher F1 scores. IForest generally maintains very high recall but suffers from low precision, flagging many normal samples as anomalies. PoAC balances precision and recall more effectively, reducing false positives while retaining strong detection of true anomalies. For instance, on *balance-scale*, IForest achieves recall of 1.0 but precision of 0.30 (F1 = 0.37), whereas PoAC improves precision to 1.0 with only a moderate drop in recall (0.72), resulting in F1 = 0.76. Similar patterns are observed across other datasets, highlighting PoAC's practical advantage in applications such as medical diagnostics, industrial quality control, and fraud detection, where accurately identifying anomalies while minimizing false alarms is critical.

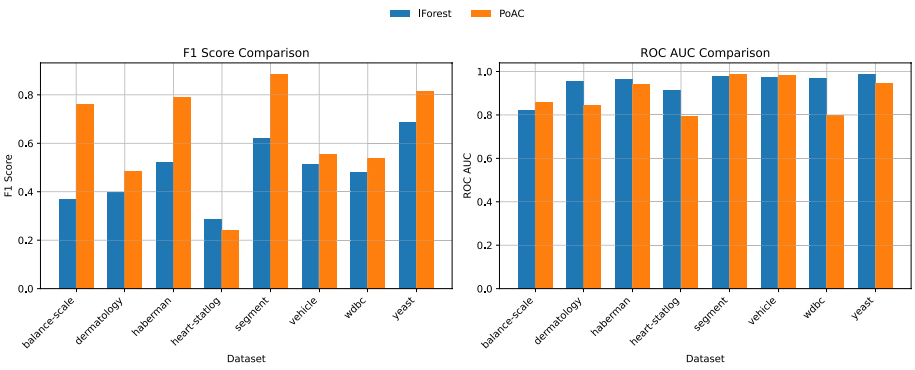


Fig. 11 Comparison between PoAC and IForest for anomaly detection on eight UCI datasets. Left: F1 score; right: ROC AUC. In this evaluation, PoAC obtains higher F1 scores on most datasets and broadly comparable ROC AUC values

Table 9 Precision, Recall, and F1 score for PoAC and IForest across the eight representative datasets

Dataset	IForest			PoAC		
	Prec.	Rec.	F1	Prec.	Rec.	F1
balance-scale	0.302	1.000	0.369	1.000	0.722	0.762
dermatology	0.273	1.000	0.397	0.392	0.963	0.485
haberman	0.429	1.000	0.521	0.711	0.889	0.789
heart-statlog	0.187	1.000	0.286	0.173	0.897	0.241
segment	0.521	0.999	0.621	0.811	0.986	0.887
vehicle	0.377	1.000	0.514	0.402	1.000	0.556
wdbc	0.349	1.000	0.482	0.491	0.608	0.539
yeast	0.576	1.000	0.687	0.789	0.905	0.816

PoAC generally achieves better balance between precision and recall, resulting in higher F1

7 Limitations

PoAC depends on the representativeness of the synthetic datasets used to train its surrogate. Distribution shifts between this problem space and a target dataset may therefore reduce predictive and pipeline quality, particularly for structures not covered by the mixture-model generator. Likewise, the choice and informativeness of the meta-features and CVIs determine which aspects of a partition the surrogate can represent. Although PoAC permits problem-specific definitions, selecting these descriptors may require domain knowledge and introduces design choices that should be validated for each use case.

The evaluation is also limited in scope. The experiments instantiate PoAC for visualization-oriented clustering and global point-anomaly detection; they do not establish performance for other unsupervised tasks or for local and contextual anomalies. The anomaly study uses IForest as its sole baseline and a fixed set of eight real-world datasets. Broader comparisons with additional anomaly detectors, data domains, and contamination mechanisms are required before generalizing those findings.

Finally, optimization quality depends on the available computational budget, while construction of the problem and meta-spaces incurs an up-front cost. Larger budgets can improve the discovered pipelines but increase runtime. The present results should therefore be interpreted under the reported search spaces, datasets, and budgets rather than as unconditional superiority across all clustering settings.

8 Conclusion

The PoAC framework introduces a flexible and modular approach to automating clustering tasks, addressing key limitations of conventional unsupervised AutoML systems. By leveraging a comprehensive meta-knowledge base of previous clustering datasets and their solutions, PoAC induces a surrogate model capable of inferring the quality of new clustering pipelines without the need for retraining or additional data ingestion. This design enables the framework to adapt to different unsupervised learning scenarios while preserving a consistent methodological foundation.

A distinctive feature of PoAC lies in its ability to define a dynamic relationship between clustering problems, CVIs, and meta-features at the framework level. This flexibility allows users to instantiate PoAC for specific tasks, such as visualization or anomaly detection,

by selecting suitable meta-features, optimization objectives, and evaluation metrics. Once instantiated, each configuration operates with fixed components, ensuring reproducibility while maintaining adaptability across problem domains. Moreover, PoAC remains algorithm-agnostic and decoupled from any specific optimization strategy, enabling seamless integration with diverse clustering algorithms and preprocessing techniques.

Within the reported visualization-oriented experiments, PoAC obtained the highest mean ARI and SIL and the lowest mean DBS among the evaluated AutoML frameworks. In the anomaly-detection use case, it obtained higher F1 scores than IForest on most of the eight datasets and broadly comparable ROC AUC values. Taken together, these results provide evidence that the same framework can be configured for two distinct objectives; they do not establish superiority beyond the evaluated datasets, baselines, search spaces, and budgets.

Future work will focus on extending PoAC's applicability to additional problem domains and exploring strategies for scalability and robustness. In particular, we aim to enhance its efficiency for large-scale and high-dimensional datasets, as well as integrate mechanisms for robust noise handling and adaptive feature selection. Such developments will further strengthen PoAC's ability to deliver reliable, context-aware, and interpretable clustering solutions across an expanding range of real-world applications.

Author Contributions Conceptualization: Matheus Camilo da Silva, Sylvio Barbon Junior; Formal analysis and investigation: Matheus Camilo da Silva, Gabriel Marques Tavares, Eric Medvet, Sylvio Barbon Junior; Writing—original draft preparation: Matheus Camilo da Silva, Gabriel Marques Tavares, Eric Medvet, Sylvio Barbon Junior; Writing—review and editing: Matheus Camilo da Silva, Gabriel Marques Tavares, Eric Medvet, Sylvio Barbon Junior; Supervision: Sylvio Barbon Junior.

Funding The author Matheus Camilo da Silva receives funding from the Ministero dell'Università e della Ricerca (Italian Ministry of University and Research) in the form of a PhD scholarship.

Code Availability The code for the execution of experiments and the analysis of results is available at https://github.com/Mcamilo/poac

Declarations

Conflict of interest The authors have no Conflict of interest to declare that are relevant to the content of this article.

Ethical Approval Not applicable.

Consent to Participate Not applicable.

Consent for Publication Not applicable.

References

Agyemang, E. F. (2024). Anomaly detection using unsupervised machine learning algorithms: A simulation study. *Scientific African,26*, Article 02386. https://doi.org/10.1016/j.sciaf.2024.e02386

Al-Jabery, K., Obafemi-Ajayi, T., Olbricht, G., & Wunsch, D. (2019). *Computational learning approaches to data analytics in biomedical applications*. Academic Press.

Aqeel, M., Sharifi, S., Cristani, M., & Setti, F. (2025). Towards real unsupervised anomaly detection via confident meta-learning. arXiv preprint arXiv:2508.02293.

Assent, I. (2012). Clustering high dimensional data. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery,2*(4), 340–350.

Bagirov, A. M., Aliguliyev, R. M., & Sultanova, N. (2023). Finding compact and well-separated clusters: Clustering using silhouette coefficients. *Pattern Recognition,135*, Article 109144. https://doi.org/10.1016/j.patcog.2022.109144

Bahri, M., Salutari, F., Putina, A., & Sozio, M. (2022). Automl: State of the art with a focus on anomaly detection, challenges, and research directions. *International Journal of Data Science and Analytics,14*(2), 113–126.

Bonner, R. E. (1964). On some clustering techniques. *IBM Journal of Research and Development,8*(1), 22–32.

Brazdil, P., Rijn, J. N., Soares, C., & Vanschoren, J. (2022). Metalearning: Applications to automated machine learning and data mining. *Springer*. https://doi.org/10.1007/978-3-030-67024-5

Breiman, L. (2001). Random forests. *Machine Learning,45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Caliński, T., & Harabasz, J. (1974). A dendrite method for cluster analysis. *Communications in Statistics-Theory and Methods,3*(1), 1–27.

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys (CSUR),41*(3), 1–58.

Chatzimparmpas, A., Martins, R. M., Jusufi, I., & Kerren, A. (2020). A survey of surveys on the use of visualization for interpreting machine learning models. *Information Visualization,19*(3), 207–233.

Chicco, D., Sabino, G., Oneto, L., & Jurman, G. (2025). The dbcv index is more informative than dcsi, cdbw, and viasckde indices for unsupervised clustering internal assessment of concave-shaped and density-based clusters. *PeerJ Computer Science,11*, Article e3095.

Cohen-Shapira, N., & Rokach, L. (2021). Automatic selection of clustering algorithms using supervised graph embedding. *Information Sciences,577*, 824–851. https://doi.org/10.1016/j.ins.2021.08.028

Davies, D. L., & Bouldin, D. W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence,2*, 224–227.

ElShawi, R., & Sakr, S. (2022). Tpe-autoclust: A tree-based pipline ensemble framework for automated clustering. In: *2022 IEEE International Conference on Data Mining Workshops (ICDMW)*, pp. 1144–1153. https://doi.org/10.1109/ICDMW58026.2022.00149

ElShawi, R., Lekunze, H., & Sakr, S. (2021). csmartml: A meta learning-based framework for automated selection and hyperparameter tuning for clustering. In: *2021 IEEE International Conference on Big Data (Big Data)*, pp. 1119–1126. https://doi.org/10.1109/BigData52589.2021.9671542

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. . (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *In: Kdd,96*, 226–231.

Ezugwu, A. E., Ikotun, A. M., Oyelade, O. O., Abualigah, L., Agushaka, J. O., Eke, C. I., & Akinyelu, A. A. (2022). A comprehensive survey of clustering algorithms: State-of-the-art machine learning applications, taxonomy, challenges, and future research prospects. *Engineering Applications of Artificial Intelligence,110*, Article 104743.

Fernandes, L., Lorena, A. C., & Smith-Miles, K. (2021). Towards understanding clustering problems and algorithms: An instance space analysis. *Algorithms*. https://doi.org/10.3390/a14030095

Ferrari, D. G., & Castro, L. N. (2012). Clustering algorithm recommendation: A meta-learning approach. In B. K. Panigrahi, S. Das, P. N. Suganthan, & P. K. Nanda (Eds.), *Swarm, evolutionary, and memetic computing* (pp. 143–150). Springer.

Ferrari, D. G., & de Castro, L. N. (2015). Clustering algorithm selection by meta-learning systems: A new distance-based problem characterization and ranking combination methods. *Information Sciences,301*, 181–194. https://doi.org/10.1016/j.ins.2014.12.044

Gabbay, I., Shapira, B., & Rokach, L. (2021). Isolation forests and landmarking-based representations for clustering algorithm recommendation using meta-learning. *Information Sciences,574*, 473–489. https://doi.org/10.1016/j.ins.2021.06.033

He, X., Zhao, K., & Chu, X. (2021). Automl: A survey of the state-of-the-art. *Knowledge-Based Systems,212*, Article 106622.

Hennig, C. (2015). What are the true clusters? *Pattern Recognition Letters,64*, 53–62. https://doi.org/10.1016/j.patrec.2015.04.009. Philosophical Aspects of Pattern Recognition

Hubert, L., & Arabie, P. (1985). Comparing partitions. *Journal of Classification,2*, 193–218.

Hutter, F., Kotthoff, L., & Vanschoren, J. (2019). *Automated machine learning: Methods, systems, challenges*. Springer.

Ilter, N., & Guvenir, H. (1998). Dermatology. *UCI Machine Learning Repository*. https://doi.org/10.24432/C5FK5P

Kryszczuk, K., & Hurley, P. (2010). Estimation of the number of clusters using multiple clustering validity indices. In: *International Workshop on Multiple Classifier Systems*. https://api.semanticscholar.org/CorpusID:17705188

Liu, Y., Li, S., & Tian, W. (2021). Autocluster: Meta-learning based ensemble method for automated unsupervised clustering. In K. Karlapalem, H. Cheng, N. Ramakrishnan, R. K. Agrawal, P. K. Reddy, J. Srivastava, & T. Chakraborty (Eds.), *Advances in knowledge discovery and data mining* (pp. 246–258). Springer.

Lopes, R. G., Yin, D., Poole, B., Gilmer, J., & Cubuk, E. D. (2019). Improving robustness without sacrificing accuracy with patch gaussian augmentation. arXiv preprint arXiv:1906.02611.

Lorena, A. C., Garcia, L. P. F., Lehmann, J., Souto, M. C. P., & Ho, T. K. (2019). How complex is your classification problem? A survey on measuring classification complexity. *ACM Computing Surveys*. https://doi.org/10.1145/3347711

Luxburg, U., Williamson, R.C., & Guyon, I. (2012). Clustering: Science or art? In: Guyon, I., Dror, G., Lemaire, V., Taylor, G., Silver, D. (Eds.), *Proceedings of ICML workshop on unsupervised and transfer learning. Proceedings of machine learning research* (Vol. 27, pp. 65–79). PMLR. https://proceedings.mlr.press/v27/luxburg12a.html

Markelle Kelly, K.N. (2017). Rachel Longjohn: UCI Machine Learning Repository. http://archive.ics.uci.edu/ml

Marutho, D., Handaka, S. H., & Wijaya, E. (2018). Muljono: The determination of cluster number at k-mean using elbow method and purity evaluation on headline news. 2018 *International Seminar on Application for Technology of Information and Communication,* pp. 533–538.

Maulik, U., & Bandyopadhyay, S. (2002). Performance evaluation of some clustering algorithms and validity indices. *IEEE Transactions on Pattern Analysis and Machine Intelligence,24*, 1650–1654.

Mejri, N., Lopez-Fuentes, L., Roy, K., Chernakov, P., Ghorbel, E., & Aouada, D. (2024). Unsupervised anomaly detection in time-series: An extensive evaluation and analysis of state-of-the-art methods. *Expert Systems with Applications,256*, Article 124922.

Mishra, S., Monath, N., Boratko, M., Kobren, A., & McCallum, A. (2022). An evaluative measure of clustering methods incorporating hyperparameter sensitivity. *Proceedings of the AAAI Conference on Artificial Intelligence,36*(7), 7788–7796. https://doi.org/10.1609/aaai.v36i7.20747

Moulavi, D., Jaskowiak, P. A., Campello, R. J., Zimek, A., & Sander, J. (2014). Density-based clustering validation. In: *Proceedings of the 2014 SIAM International Conference on Data Mining* (pp. 839–847). SIAM.

Nascimento, A. C. A., Prudêncio, R. B. C., Souto, M. C. P., & Costa, I. G. (2009). Mining rules for the automatic selection process of clustering methods applied to cancer gene expression data. In C. Alippi, M. Polycarpou, C. Panayiotou, & G. Ellinas (Eds.), *Artificial Neural Networks - ICANN 2009* (pp. 20–29). Springer.

Nemenyi, P. B. (1963). *Distribution-free multiple comparisons*. Princeton University.

Olson, R. S., & Moore, J. H. (2016). Tpot: A tree-based pipeline optimization tool for automating machine learning. *Workshop on Automatic Machine Learning* (pp. 66–74) PMLR.

Olson, R.S., Urbanowicz, R.J., Andrews, P.C., Lavender, N.A., Kidd, L.C., & Moore, J.H. (2016). Automating biomedical data science through tree-based pipeline optimization. In: EvoApplications. https://api.semanticscholar.org/CorpusID:9709316

Pimentel, B. A., & de Carvalho, A. C. P. L. F. (2019). A new data characterization for selecting clustering algorithms using meta-learning. *Information Sciences,477*, 203–219. https://doi.org/10.1016/j.ins.2018.10.043

Pimentel, B.A., & Carvalho, A.C.P.L.F. (2018). Statistical versus distance-based meta-features for clustering algorithm recommendation using meta-learning. In: *2018 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8. https://doi.org/10.1109/IJCNN.2018.8489182

Pimentel, B.A., & Carvalho, A.C.P.L.F. (2019). Unsupervised meta-learning for clustering algorithm recommendation. In: *2019 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8 https://doi.org/10.1109/IJCNN.2019.8851989

Poulakis, Y., Doulkeridis, C., & Kyriazis, D. (2020). Autoclust: A framework for automated clustering based on cluster validity indices. *2020 IEEE International Conference on Data Mining (ICDM)* (pp. 1220–1225). IEEE.

Rice, J. R. (1976). *The algorithm selection problem*. Elsevier.

Rivolli, A., Garcia, L. P., Soares, C., Vanschoren, J., & Carvalho, A. C. (2018). Characterizing classification datasets: a study of meta-features for meta-learning. arXiv preprint arXiv:1808.10406.

Rivolli, A., Garcia, L. P., Soares, C., Vanschoren, J., & Carvalho, A. C. (2022). Meta-features for meta-learning. *Knowledge-Based Systems,240*, Article 108101.

Rokach, L., & Maimon, O. (2005). Clustering methods. *Data mining and knowledge discovery handbook* (pp. 321–352)

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of computational and applied mathematics,20*, 53–65.
Shahapure, K. R., & Nicholas, C. K. (2020). Cluster quality analysis using silhouette score. *IEEE 7th International Conference on Data Science and Advanced Analytics (DSAA)* (pp. 747–748)
Silva, M.C., Licari, B., Tavares, G.M., & Junior, S.B. (2024). Benchmarking autoML clustering frameworks. In: AutoML Conference 2024 (ABCD Track). https://openreview.net/forum?id=RzUKJnph1g
Soares, R. G. F., Ludermir, T. B., & De Carvalho, F. A. T. (2009). An analysis of meta-learning techniques for ranking clustering algorithms applied to artificial data. In C. Alippi, M. Polycarpou, C. Panayiotou, & G. Ellinas (Eds.), *Artificial Neural Networks - ICANN 2009* (pp. 131–140). Springer.
Souto, M.C.P., Prudencio, R.B.C., Soares, R.G.F., Araujo, D.S.A., Costa, I.G., Ludermir, T.B., & Schliep, A. (2008). Ranking and selecting clustering algorithms using a meta-learning approach. In: *2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*, pp. 3729–3735. https://doi.org/10.1109/IJCNN.2008.4634333
Strehl, A., & Ghosh, J. (2003). Relationship-based clustering and visualization for high-dimensional data mining. *INFORMS Journal on Computing,15*(2), 208–230.
Thomas, J. C. R., Peñas, M. S., & Mora, M. (2013). New version of davies-bouldin index for clustering validation based on cylindrical distance. *32nd International Conference of the Chilean Computer Science Society (SCCC)* (pp. 49–53)
Treder-Tschechlov, D., Fritz, M., Schwarz, H., & Mitschang, B. (2023). Ml2dac: Meta-learning to democratize automl for clustering analysis. *Proceedings of the ACM on Management of Data,1*(2), 1–26.
Truong, A., Walters, A., Goodsitt, J., Hines, K., Bruss, C. B., & Farivar, R. (2019). Towards automated machine learning: Evaluation and comparison of automl approaches and tools. *IEEE 31st International Conference on Tools with Artificial Intelligence (ICTAI)* (pp. 1471–1479). IEEE.
Tschechlov, D., Fritz, M., & Schwarz, H. (2021). AutoML4Clust: Efficient AutoML for clustering analyses. OpenProceedings.org. https://doi.org/10.5441/002/EDBT.2021.32. https://openproceedings.org/2021/conf/edbt/p87.pdf
Van Craenendonck, T., & Blockeel, H. (2017). Constraint-based clustering selection. *Machine Learning,106*(9), 1497–1521. https://doi.org/10.1007/s10994-017-5643-7
Van Mechelen, I., Boulesteix, A.-L., Dangl, R., Dean, N., Hennig, C., Leisch, F., Steinley, D., & Warrens, M. J. (2023). A white paper on good research practices in benchmarking: The case of cluster analysis. *WIREs Data Mining and Knowledge Discovery,13*(6), 1511. https://doi.org/10.1002/widm.1511
Vanschoren, J. (2018). Meta-learning: A survey. arXiv preprint arXiv:1810.03548.
Yao, Q., Wang, M., Chen, Y., Dai, W., Li, Y.-F., Tu, W.-W., Yang, Q., & Yu, Y. (2019). Taking human out of learning applications: A survey on automated machine learning. arXiv:1810.13306 [cs.AI]
Zellinger, M. J., & Bühlmann, P. (2023). repliclust: Synthetic data for cluster analysis
Zöller, M.-A., & Huber, M. F. (2021). Benchmark and survey of automated machine learning frameworks. *Journal of Artificial Intelligence Research,70*, 409–472. https://doi.org/10.1613/jair.1.11854

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Matheus Camilo da Silva[1,2] · Gabriel Marques Tavares[3,4] · Eric Medvet[2] · Sylvio Barbon Junior[2]

✉ Matheus Camilo da Silva
matheus.camilodasilva@phd.units.it

✉ Sylvio Barbon Junior
sbarbonjr@gmail.com

Gabriel Marques Tavares
tavares@dbs.ifi.lmu.de

Eric Medvet
emedvet@units.it

[1] Applied Data Science & Artificial Intelligence, University of Trieste, Piazzale Europa, 1, 34127 Trieste, Friuli-Venezia Giulia, Italy

[2] Department of Engineering and Architecture, University of Trieste, Piazzale Europa, 1, 34127 Trieste, Friuli-Venezia Giulia, Italy

[3] Database Systems and Data Mining, LMU Munich, Oettingenstraße 67, 80538 Munich, Bavaria, Germany

[4] Munich Center for Machine Learning (MCML), Munich, Bavaria, Germany